Министерство науки и высшего образования Российской Федерации

Федеральное государственное автономное образовательное
учреждение высшего образования
Национальный исследовательский Нижегородский государственный
университет им. Н.И. Лобачевского

Институт информационных технологий, математики и механики

Отчет по лабораторной работе №2

«Таблицы»

Выполнил:
студент группы 3824Б1ФИ2
Ципин Д. Д.

Проверил:
Ивлев А.Д.

Нижний Новгород
2026

Содержание

Введение 3
1. Постановка задачи 4
2. Руководство пользователя 6
2.1. Запуск программы и главное меню 6
2.2. Добавление данных 6
2.3. Поиск данных 7
2.4. Удаление данных 8
2.5. Просмотр результатов и лог-файла 8
3. Руководство программиста 10
3.1. Описание структуры программы 10
3.2. Описание структур данных 11
3.3. Описание алгоритмов 13
4. Результаты экспериментов и тестирования 16
4.1. Google tests 16
4.2. Результаты экспериментов 17
Заключение 19
Литература 20

Введение

В программировании одной из частых задач является хранение и поиск информации по какому-то имени или ключу. Существует много способов организовать такое хранение: от простых списков до деревьев и хэш-таблиц. Каждый из этих способов имеет свои плюсы и минусы: одни быстрее ищут, другие быстрее добавляют новые данные.

В данной работе мы рассматриваем, как выбор структуры данных влияет на производительность программы. Чтобы это проверить, мы реализуем несколько видов таблиц и внедрим в них систему подсчета каждой операции. Это позволит получить точные цифры: сколько сравнений данных и присваиваний потребовалось для каждого действия.

1. Постановка задачи

Цель работы – разработать программу, которая предоставляет возможность хранить данные в таблицах трёх типов с подсчётом количества произведённых операций с момента создания:

1. Неупорядоченная таблица (на векторе)
2. Упорядоченная таблица (на векторе)
3. Упорядоченная таблица на сбалансированном бинарном дереве поиска – АВЛ дерево
4. Хэш-таблица
5. Организовать синхронную работу со всеми таблицами: любое действие пользователя (вставка, поиск, удаление) должно выполняться одновременно во всех структурах.

Технические требования и особенности реализации:

1. **Параллельная работа со структурами данных**: Программа обеспечивает одновременное взаимодействие со всеми типами реализованных таблиц при выполнении любой операции.
2. **Типизация ключей**: В качестве ключа используется строка. Сравнение ключей для определения порядка в упорядоченных структурах производится лексикографически.
3. **Механизм хэширования**:
   - Хэш-таблица реализует метод цепочек для борьбы с коллизиями
   - Применяется хэш-функция, удовлетворяющая условиям равномерного распределения ключей (*алгоритм DJB2*).
4. **Пользовательский ввод**: Предусмотрена возможность сохранения данных пользователя в таблицу с присвоением им ключа.
5. **Доступ к данным**: Реализован функционал поиска и извлечения данных из таблиц по заданному пользователем ключу.

6. **Приложение**: Консольное приложение поддерживает работу с полиномами из Лабораторной работы №1. Пользователь может производить над ними арифметические операции и размещать полученные результаты в таблицы.
7. **Система логирования**:
    - В процессе работы приложения осуществляется автоматический подсчет и логирование количества произведенных операций для каждой таблицы.
    - Для каждого действия (вставка, поиск, извлечение) в отдельный файл записывается тип и количество вызовов операции для разных типов таблиц.
    - После завершения работы приложения в файл логирования добавляется итоговая сводка: общее число операций для каждого действия и суммарное количество операций по каждому типу таблиц за весь сеанс.

2. Руководство пользователя

Данный раздел описывает порядок работы с консольным приложением для управления таблицами данных. Программа позволяет сохранять полиномы под уникальными именами, искать их и удалять, одновременно отслеживая эффективность алгоритмов в фоновом режиме.

2.1. Запуск программы и главное меню

При запуске приложения пользователь видит главное текстовое меню. Навигация осуществляется путем ввода цифры, соответствующей нужному пункту, и нажатия клавиши **Enter**.

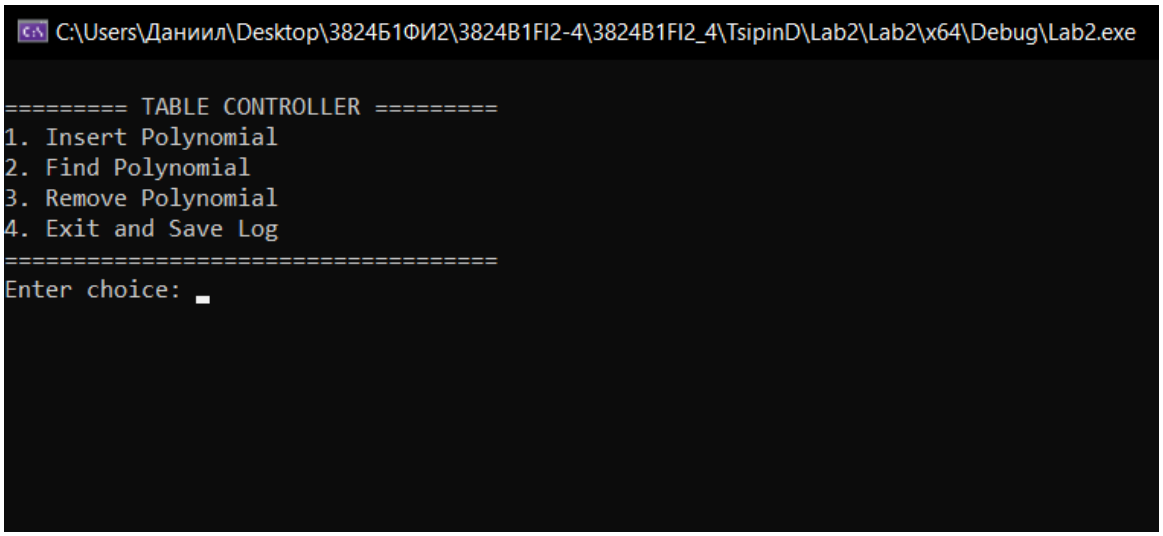

Рисунок 1. Меню команд

Доступные пункты меню:

1. **Insert Polynomial** – добавление нового полинома во все таблицы.
2. **Find Polynomial** – поиск существующего полинома по его имени.
3. **Remove Polynomial** – удаление записи из всех таблиц.
4. **Exit and Save Log** – завершение работы и сохранение итоговой статистики.

2.2. Добавление данных

Для добавления полинома выберите пункт **«1»**.

1. Программа предложит ввести уникальный ключ (имя). Ключ должен состоять из одного слова (например, Poly1, A, result).

2. Далее необходимо ввести мономы полинома в формате: коэффициент степень.

3. Для завершения ввода полинома необходимо нажать **Enter** на пустой строке.

Пример ввода:

```
C:\Users\Даниил\Desktop\3824Б1ФИ2\3824B1FI2-4\3824B1FI2_4\TsipinD\Lab2\Lab2\x64\Debug\Lab2.exe

========= TABLE CONTROLLER =========
1. Insert Polynomial
2. Find Polynomial
3. Remove Polynomial
4. Exit and Save Log
====================================
Enter choice: 1
Enter unique key (for example 'A' or 'Poly1'): Poly1
Enter the monomials in the format: <coefficient> <degree>. An empty line is the end of the input.
-3.5 101
2 100
10 3

[System] Inserted successfully.
```

Рисунок 2. Пример вставки полинома

После этого полином будет автоматически добавлен во все четыре таблицы, а информация об операциях (сравнениях и присваиваниях) будет отправлена в файл логирования.

2.3. Поиск данных

Для поиска ранее сохраненного полинома выберите пункт **«2»**.

1. Введите имя (ключ), который вы указывали при добавлении.
2. Если ключ найден, программа выведет его математический вид на экран.
3. Если ключа нет, приложение сообщит: *[Error] Key not found*.

```
C:\Users\Даниил\Desktop\3824Б1ФИ2\3824B1FI2-4\38

========= TABLE CONTROLLER =========
1. Insert Polynomial
2. Find Polynomial
3. Remove Polynomial
4. Exit and Save Log
====================================
Enter choice: 2
Enter key to find: Poly1
[Found] Poly1 = 10 3 , -3.5 101 , 2 100 ,
```

Рисунок 4. Полином найден

```
C:\Users\Даниил\Desktop\3824Б1ФИ2\3824B1FI2-4\3824

========= TABLE CONTROLLER =========
1. Insert Polynomial
2. Find Polynomial
3. Remove Polynomial
4. Exit and Save Log
====================================
Enter choice: 2
Enter key to find: Poly2
[Error] Key 'Poly2' not found.
```

Рисунок 3. Полином не найден

Поиск происходит во всех таблицах параллельно, что позволит сравнить скорость доступа по результатам количества операций в файле логирования.

2.4. Удаление данных

Для удаления записи выберите пункт **«3»**.

1. Введите ключ удаляемого элемента.
2. Программа произведет удаление из всех структур данных.
3. В лог-файл будет записана статистика: сколько операций потребовалось каждой таблице для перестройки своей структуры.

```
C:\Users\Даниил\Desktop\3824Б1ФИ2\3824B1FI2-4\3824B1FI2_4\TsipinD\Lab2\Lab2\x64\Debug\Lab2.exe

========= TABLE CONTROLLER =========
1. Insert Polynomial
2. Find Polynomial
3. Remove Polynomial
4. Exit and Save Log
====================================
Enter choice: 3
Enter key to remove: Poly1
[System] Key 'Poly1' removed from all tables.
```

Рисунок 5. Удаление полинома

2.5. Просмотр результатов и лог-файла

Пользователь может в любой момент открыть текстовый файл lab_log.txt (в папке с программой), чтобы изучить эффективность работы таблиц.

В файле отображается:

- Какое действие было выполнено.
- Количество сравнений на равенство (equals).
- Количество сравнений на порядок (less).
- Количество операций присваивания (assignments).

При выборе пункта **«4» (Exit)** программа автоматически дописывает в конец файла итоговую таблицу с суммарными затратами ресурсов за весь сеанс работы.

lab_log – Блокнот

Файл Правка Формат Вид Справка

```
=== Table Operations Log ===

Operation: INSERT | Key: Poly1
Unordered    -> Equals: 0     | Less: 0     | Assign: 1
Ordered      -> Equals: 0     | Less: 0     | Assign: 1
AVL Tree     -> Equals: 0     | Less: 0     | Assign: 1
Hash Table   -> Equals: 0     | Less: 0     | Assign: 1
--------------------------------------------------
Operation: FIND | Key: Poly1
Unordered    -> Equals: 1     | Less: 0     | Assign: 0
Ordered      -> Equals: 1     | Less: 0     | Assign: 0
AVL Tree     -> Equals: 1     | Less: 0     | Assign: 0
Hash Table   -> Equals: 1     | Less: 0     | Assign: 0
--------------------------------------------------
Operation: FIND | Key: Poly2
Unordered    -> Equals: 1     | Less: 0     | Assign: 0
Ordered      -> Equals: 1     | Less: 1     | Assign: 0
AVL Tree     -> Equals: 1     | Less: 1     | Assign: 0
Hash Table   -> Equals: 0     | Less: 0     | Assign: 0
--------------------------------------------------
Operation: REMOVE | Key: Poly1
Unordered    -> Equals: 1     | Less: 0     | Assign: 2
Ordered      -> Equals: 1     | Less: 0     | Assign: 1
AVL Tree     -> Equals: 1     | Less: 0     | Assign: 1
Hash Table   -> Equals: 1     | Less: 0     | Assign: 0
--------------------------------------------------

=== FINAL SUMMARY (Total Operations) ===
Unordered Table: Total Equals=3, Total Less=0, Total Assign=3
Ordered Table: Total Equals=3, Total Less=1, Total Assign=2
AVL Tree Table: Total Equals=3, Total Less=1, Total Assign=2
Hash Table: Total Equals=2, Total Less=0, Total Assign=1
```

Рисунок 6. Файл логирования

3. Руководство программиста

3.1. Описание структуры программы

Основная архитектура разделена на модули хранения данных, аналитический модуль, модуль предметной области и пользовательский интерфейс.

1. Модуль структур данных (Таблицы)

Этот модуль является ядром программы и содержит реализации четырех типов таблиц. Все классы реализованы как шаблоны (*template <typename T>*), что позволяет им хранить любые данные, используя *std::string* в качестве ключа.

- **UnorderedTable.h**: содержит класс **UnorderedTable**. Реализует простейшую таблицу на основе вектора.
- **OrderedTable.h**: содержит класс **OrderedTable**. Реализует упорядоченную таблицу на основе отсортированного вектора.
- **AVLTreeTable.h**: содержит класс **AVLTreeTable**. Реализует сбалансированное дерево поиска. Содержит логику поворотов для поддержания высоты дерева.
- **HashTable.h**: содержит класс **HashTable**. Реализует хэш-таблицу с методом цепочек. Включает в себя внутреннюю хэш-функцию *DJB2*.

2. Аналитический модуль (Статистика и Логирование)

Предназначен для сбора данных о производительности алгоритмов.

- **OpStats.h**: содержит структуру **OpStats**. Служит для хранения счетчиков операций: сравнений на больше/меньше, сравнений на равенство и операций присваивания. Включает перегруженные операторы + и += для суммирования статистики.

- **Logger.h**: содержит класс **Logger**. Отвечает за запись результатов в текстовый файл. Основная функция – **LogOperation**, которая принимает текущую статистику от всех таблиц и записывает её в читаемом виде.

3. Модуль предметной области (Полиномы)

Интегрирует код из предыдущей лабораторной работы (Блок 1).

- **polynomial.h и list.h**: содержат классы **Polynom**, **List** и структуру **Node**. Реализуют представление математического полинома в виде связного списка и базовые операции над ними.

4. Модуль управления и интерфейса

Связывает все компоненты в пользовательском приложении.

- **main.cpp**: содержит функции управления приложением.
    - **SyncInsert**, **SyncFind**, **SyncRemove** – функции-синхронизаторы, которые вызывают соответствующие методы сразу у всех пяти таблиц.
    - Главный цикл **main** – обрабатывает пользовательское меню и ввод данных.

5. Модуль тестирования

test_tables.cpp: содержит набор тестов на базе фреймворка **Google Test**. Проверяет корректность работы каждой таблицы в отдельности, включая тесты со случайными ключами и значениями.

3.2. Описание структур данных

1. Структура OpStats (Статистика)

Служит для детального мониторинга ресурсов.

- **Поля**: equals (сравнения ==), comparisons (сравнения <), assignments (копирование объектов).

2. Класс UnorderedTable<T> (Неупорядоченная таблица)

Реализована на базе динамического массива std::vector<std::pair<std::string, T>>.

- **Поля**: data (вектор пар «ключ-значение»), stats (объект статистики).
- **Временная сложность**:
  - Find: O(n) – в худшем случае нужно просмотреть весь массив.
  - Insert: O(n) – так как по условию мы должны проверить уникальность ключа перед вставкой (поиск дубликата занимает O(n)).
  - Remove: O(n) – поиск элемента занимает O(n), само удаление (замена на последний элемент) – O(1).
- **Память**:

O(n) для хранения элементов. Дополнительная память при операциях — O(1).

3. Класс OrderedTable<T> (Упорядоченная таблица)

Использует отсортированный по ключу вектор std::vector<std::pair<std::string, T>>.

- **Временная сложность**:
  - Find: O(log n) – используется бинарный поиск. Каждый шаг делит область поиска пополам.
  - Insert: O(n) – поиск места вставки занимает O(log n), но вставка в середину массива требует сдвига всех последующих элементов вправо, что в среднем дает n/2 операций.
  - Remove: O(n) – аналогично вставке, требуется сдвиг элементов влево.
- **Память**:

O(n) для хранения элементов.

4. Класс AVLTreeTable<T> (Сбалансированное АВЛ-дерево)

Хранит данные в виде дерева узлов Node, где каждый узел содержит ссылки на детей, ключ, значение и высоту.

- **Структура Node**: key, value, height, left*, right*.
- **Временная сложность**:
  - Find, Insert, Remove: O(log n) всегда.

- **Вывод**: Высота АВЛ-дерева строго ограничена формулой **H<1.44log2(n+2)**. Благодаря поворотам при вставке и удалении, дерево никогда не вырождается в список, что гарантирует логарифмическую сложность.

- **Память**:

O(n). Каждый элемент обернут в структуру Node.

- **Дополнительная память**:

O(H) при рекурсивных вызовах (стек вызовов).

5. Класс HashTable<T> (Хэш-таблица)

Использует массив списков (std::vector<std::list<...>>).

- **Поля**: buckets (вектор корзин), bucketCount (размер массива).

- **Временная сложность**:

  - Find, Insert, Remove: В среднем O(1+a), где a=n/k (a – коэффициент заполнения, k – число корзин).

  - Если хэш-функция распределяет ключи равномерно, длина каждой цепочки будет близка к n/k. В худшем случае (все ключи дали один хэш и попали в одну корзину) сложность равна O(n).

- **Память**:

O(n+k), где n – число элементов, k – число корзин.

3.3. Описание алгоритмов

1. Алгоритм бинарного поиска (для упорядоченной таблицы)

Применяется в OrderedTable для поиска элемента и определения позиции вставки.

- **Логика**: Алгоритм работает на отсортированном массиве. На каждом шаге выбирается серединный элемент (mid). Если искомый ключ меньше ключа в mid, поиск продолжается в левой половине, если больше — в правой.

- **Особенность реализации**: Наша реализация возвращает индекс элемента, если он найден, или индекс «ближайшего большего» элемента, если ключ отсутствует. Это позволяет методу Insert знать точное место для вставки нового узла без повторного сканирования массива.

- **Сложность**: O(log n) по времени.

2. Алгоритмы балансировки АВЛ-дерева

АВЛ-дерево поддерживает свою сбалансированность за счет того, что разница высот левого и правого поддеревьев любого узла не превышает 1.

- **Вычисление баланса**: После каждой вставки или удаления программа рекурсивно поднимается к корню, обновляя высоту узлов (height = max(h_left, h_right) + 1) и проверяя фактор баланса (bal = h_right - h_left).
- **Вращения**:
  - **Малое правое/левое вращение**: В ходе вращения сын становится отцом, а отец становится сыном. Указатели перебрасываются так, чтобы сохранить свойства дерева поиска.
  - **Большое вращение (левое-правое или правое-левое)**: Применяется при «зигзагообразном» перекосе. Состоит из двух этапов: сначала малого вращения вокруг дочернего узла для превращения зигзага в линию, затем малого вращения вокруг текущего узла.
- **Сложность**: Поиск места вставки – O(log n). Сама балансировка требует не более двух поворотов при вставке и O(log n) при удалении.
- **Дополнительная память**: O(log n) на стек вызовов из-за рекурсивной работы алгоритма.

3. Алгоритм хэширования DJB2 и метод цепочек

Хэш-таблица преобразует строковый ключ в индекс массива для обеспечения доступа за константное время.

- **Функция DJB2**: Был выбран классический алгоритм Дэна Бернштейна. Его логика:
  1. Начальное значение хэша — 5381 (прайм-число).
  2. Для каждого символа строки: hash = ((hash << 5) + hash) + char (что эквивалентно hash * 33 + char).
  - Множитель 33 и начальное число 5381 минимизируют количество коллизий на строковых данных (именах, названиях).
- **Разрешение коллизий (Метод цепочек)**: Если два разных ключа дают один индекс, они помещаются в связный список (std::list), привязанный к ячейке массива (корзине).

- **Сложность**: O(L) на вычисление хэша (где L – длина строки). Поиск в цепочке в среднем занимает O(1) при достаточном размере массива и хорошем распределении.

4. Алгоритм синхронизации и логирования

Поскольку условие требует работы сразу со всеми таблицами, в **main.cpp** реализован алгоритм «Параллельного вызова»:

1. Пользователь вводит команду.
2. Для каждой из 4 таблиц последовательно:
    - Вызывается ResetStats() (обнуление счетчиков).
    - Выполняется целевой метод (Insert, Find или Remove).
    - Текущая статистика копируется в объект логгера.
3. Логгер форматирует данные и записывает их в файл lab_log.txt.

4. Результаты экспериментов и тестирования

4.1. Google tests

Для подтверждения правильности работы алгоритмов был разработан пакет из **26 автоматических тестов** на базе фреймворка **Google Test**. Проверки охватили все реализованные структуры данных и их специфические особенности.

Краткое описание набора тестов:

1. **Неупорядоченная таблица (5 тестов)**:
   - Подтверждена корректность вставки, поиска и удаления.
   - С помощью теста *UnordStatsCounting* доказано, что поиск является линейным (O(N)), а операции сравнения на порядок (comparisons/ «больше-меньше») не используются (равны 0).
2. **Упорядоченная таблица (6 тестов)**:
   - Проверено сохранение лексикографического порядка ключей.
   - Тестами *OrdStatsCounting* подтверждена логарифмическая сложность поиска: для 50 элементов количество сравнений не превышает 7, а для 5 элементов — не более 4.
3. **АВЛ-дерево (8 тестов)**:
   - **Балансировка**: тесты *BalancingAfterSequentialInsert* и *BalancingAfterRemove* подтвердили математическую точность высоты дерева (высота 3 для 7 узлов и высота 2 для 3 узлов).
   - Успешно пройден тест на **RL-поворот** и удаление корневого узла.
   - Тест *AVLOrdStatsCountingRandom* показал, что поиск среди 100 элементов требует не более 7 сравнений.
4. **Хэш-таблица (7 тестов)**:
   - **Коллизии**: тесты *CollisionTest* и *RandomCollisionTest* подтвердили работоспособность метода цепочек даже при малом количестве корзин.
   - **Аналитика**: тест *DetailedStats* подтвердил, что при удачном хэшировании поиск выполняется за 1 сравнение (equals = 1).

Все 26 тестов завершились успешно.

4.2. Результаты экспериментов

В данном разделе представлены результаты серии экспериментов, направленных на сравнение эффективности четырех типов таблиц при различных объемах данных (N=10,100,1000). Эксперимент представлен в проекте **Experiments**, в файле **experiments.cpp**.

Методика эксперимента:

Для каждого значения N проводилось три этапа:

1. **INSERT**: Последовательная вставка N случайных полиномов со случайными ключами.
2. **FIND**: Поиск всех N вставленных ключей.
3. **REMOVE**: Последовательное удаление всех N ключей.

Замеры производились по количеству операций сравнения на равенство (*Equals*), сравнения на порядок (*Comparisons*), присваиваний (*Assign*) и по физическому времени выполнения в миллисекундах.

```
Консоль отладки Microsoft Visual Studio

>>>>>>>>> STARTING EXPERIMENT FOR N = 10 ELEMENTS <<<<<<<<<

STAGE: INSERT
-------------------------------------------------------------------------------------
| Table Type         | Equals (==)     | Comparisons (><) | Assign (=)      | Time (ms)  |
-------------------------------------------------------------------------------------
| Unordered          | 45              | 0                | 10              | 0.068      |
| Ordered            | 23              | 23               | 29              | 0.072      |
| AVL Tree           | 24              | 28               | 30              | 0.023      |
| Hash Table         | 1               | 0                | 10              | 0.038      |
=====================================================================================

STAGE: FIND
-------------------------------------------------------------------------------------
| Table Type         | Equals (==)     | Comparisons (><) | Assign (=)      | Time (ms)  |
-------------------------------------------------------------------------------------
| Unordered          | 55              | 0                | 0               | 0.003      |
| Ordered            | 29              | 19               | 0               | 0.003      |
| AVL Tree           | 30              | 20               | 0               | 0.003      |
| Hash Table         | 11              | 0                | 0               | 0.004      |
=====================================================================================

STAGE: REMOVE
-------------------------------------------------------------------------------------
| Table Type         | Equals (==)     | Comparisons (><) | Assign (=)      | Time (ms)  |
-------------------------------------------------------------------------------------
| Unordered          | 30              | 0                | 20              | 0.018      |
| Ordered            | 20              | 10               | 36              | 0.033      |
| AVL Tree           | 19              | 11               | 26              | 0.011      |
| Hash Table         | 10              | 0                | 0               | 0.016      |
=====================================================================================
```

Рисунок 7. Результаты эксперимента при N=10

```
Консоль отладки Microsoft Visual Studio
>>>>>>>>> STARTING EXPERIMENT FOR N = 100 ELEMENTS <<<<<<<<<

STAGE: INSERT
-------------------------------------------------------------------------------------
| Table Type         | Equals (==)     | Comparisons (><) | Assign (=)      | Time (ms)  |
-------------------------------------------------------------------------------------
| Unordered          | 4950            | 0                | 100             | 0.601      |
| Ordered            | 534             | 534              | 2358            | 1.912      |
| AVL Tree           | 544             | 592              | 412             | 0.225      |
| Hash Table         | 20              | 0                | 100             | 0.226      |
=====================================================================================

STAGE: FIND
-------------------------------------------------------------------------------------
| Table Type         | Equals (==)     | Comparisons (><) | Assign (=)      | Time (ms)  |
-------------------------------------------------------------------------------------
| Unordered          | 5050            | 0                | 0               | 0.137      |
| Ordered            | 580             | 480              | 0               | 0.042      |
| AVL Tree           | 592             | 492              | 0               | 0.039      |
| Hash Table         | 120             | 0                | 0               | 0.023      |
=====================================================================================

STAGE: REMOVE
-------------------------------------------------------------------------------------
| Table Type         | Equals (==)     | Comparisons (><) | Assign (=)      | Time (ms)  |
-------------------------------------------------------------------------------------
| Unordered          | 2550            | 0                | 200             | 0.304      |
| Ordered            | 467             | 367              | 2792            | 2.129      |
| AVL Tree           | 428             | 350              | 273             | 0.114      |
| Hash Table         | 100             | 0                | 0               | 0.135      |
=====================================================================================
```

Рисунок 9. Результаты эксперимента при N=100

```
>>>>>>>>> STARTING EXPERIMENT FOR N = 1000 ELEMENTS <<<<<<<<<

STAGE: INSERT
-------------------------------------------------------------------------------------
| Table Type         | Equals (==)     | Comparisons (><) | Assign (=)      | Time (ms)  |
-------------------------------------------------------------------------------------
| Unordered          | 499500          | 0                | 1000            | 17.933     |
| Ordered            | 8595            | 8595             | 249375          | 177.092    |
| AVL Tree           | 8768            | 9247             | 3876            | 2.622      |
| Hash Table         | 263             | 0                | 1000            | 2.408      |
=====================================================================================

STAGE: FIND
-------------------------------------------------------------------------------------
| Table Type         | Equals (==)     | Comparisons (><) | Assign (=)      | Time (ms)  |
-------------------------------------------------------------------------------------
| Unordered          | 500500          | 0                | 0               | 13.763     |
| Ordered            | 8987            | 7987             | 0               | 0.626      |
| AVL Tree           | 9211            | 8211             | 0               | 0.577      |
| Hash Table         | 1263            | 0                | 0               | 0.217      |
=====================================================================================

STAGE: REMOVE
-------------------------------------------------------------------------------------
| Table Type         | Equals (==)     | Comparisons (><) | Assign (=)      | Time (ms)  |
-------------------------------------------------------------------------------------
| Unordered          | 250500          | 0                | 2000            | 9.648      |
| Ordered            | 7621            | 6621             | 252125          | 173.323    |
| AVL Tree           | 7382            | 6655             | 3056            | 1.445      |
| Hash Table         | 1000            | 0                | 0               | 1.376      |
=====================================================================================
```

Рисунок 8. Результаты эксперимента при N=1000

Экспериментально доказано, что для большинства практических задач наиболее эффективными являются **хэш-таблицы** и **АВЛ-деревья**. Применение **упорядоченных** и **неупорядоченных таблиц** на больших объемах данных (N≥1000) приводит к падению производительности приложения и неоправданному расходу процессорного времени на перемещение объектов в памяти.

Заключение

В ходе выполнения лабораторной работы был разработан программный комплекс для эффективного хранения и управления данными на примере полиномов от трех переменных. Были реализованы четыре типа структур данных: неупорядоченная таблица на векторе, упорядоченная таблица на массиве, сбалансированное АВЛ-дерево и хэш-таблица с методом цепочек.

Основные результаты работы:

1. **Сравнительный анализ**: Экспериментально подтверждена высокая эффективность хэш-таблиц (константное время доступа) и АВЛ-деревьев (логарифмическое время). Установлено, что использование массивов для хранения больших объемов динамических данных нецелесообразно.
2. **Мониторинг производительности**: Реализованная система подсчета операций позволила детально изучить поведение алгоритмов.
3. **Корректность**: Разработаны из 26 тестов на базе Google Test, подтверждена стабильность работы всех структур, включая сложные сценарии балансировки дерева и разрешения коллизий в хэш-таблице.

Программа полностью соответствует техническому заданию, обеспечивает синхронное логирование операций во всех таблицах и предоставляет пользователю удобный интерфейс для работы с полиномами.

Литература

1. Сайт cppreference: https://en.cppreference.com/
2. YouTube, «Как работают деревья | Структуры данных», Автор: Alek OS.